Part I: HarborView Analytics CI/CD Analysis

Student: Jonathan Cantu

Professor: Richard Krasso

Date: August 30, 2026

1. Identify CI/CD Issues

HarborView Analytics has adopted CI/CD, but the current pipeline is not providing the consistency or confidence that CI/CD is supposed to create. The inconsistent build failures are especially concerning because developers cannot easily tell whether a failure represents an actual code problem or an unstable pipeline. When developers lose trust in the build process, they tend to ignore failures, rerun jobs, or work around the pipeline instead of treating failures a s useful

The long-running tests create another bottleneck. Cl depends on fast feedback, and the Week 3 reading points out that when tests take too long, developers stop paying attention to them and may begin batching changes or skipping tests altogether (Krasso, 2026). HarborView i s already seeing this behavior because developers frequently skip testing locally.

The manual staging deployment process creates additional inconsistency. Manual steps depend on people remembering the correct sequence every time, increasing the possibility of mistakes and making releases harder to reproduce. Finally, the team has no defined response when a pipeline fails. This creates uncertainty about who owns the failure, how quickly i t should be addressed, and whether work should continue while the pipeline remains broken.

2. Improve CI Practices

The 1st priority should be making Cl fast, reliable, and predictable. Every change should trigger the same automated build and test process in a consistent environment. Standardized build images or containerized environments could help reduce situations where code behaves differently locally than it does in Cl.

I would also establish the expectation that a broken pipeline is treated a s a priority rather than something the team works around. The Week 3 reading emphasizes that Cl requires cultural discipline as well as automation and that teams should address broken builds immediately (Krasso, 2026). A clear ownership process could identify who investigates a failure and what happens when the main build is unhealthy.

Pipeline performance should also be reviewed regularly. Slow stages could potentially be improved by separating fast unit tests from slower integration or end-to-end tests, running independent tests in parallel, and caching dependencies where appropriate. The goal should be to return meaningful feedback to developers a s quickly a s possible.

3. Improve CD Practices

HarborView should remove the manual steps currently required to deploy to the staging environment. Continuous Delivery depends on repeatable and predictable deployment processes. Builds should create a consistent deployable artifact, and that same artifact should move through test and staging environments rather than being rebuilt differently for each environment.

An automated staging deployment could run after required C I checks succeed. The pipeline could package the application, deploy it to staging, run additional validation, and clearly report whether the deployment succeeded or failed.

This would make releases less stressful because the team would no longer depend on a person remembering a deployment checklist. The Week 3 reading describes low-risk releases as relying on smaller changes, automated checks, consistent environments, predictable deployment steps, and fast feedback loops (Krasso, 2026).

4. Strengthen Automated Testing

HarborView's testing process needs to become both faster and more trustworthy. Automated tests should run consistently whenever code is integrated, and failures should stop the pipeline before broken code progresses farther.

I would organize the test suite so that fast unit tests provide immediate feedback, followed by integration tests that validate interactions between components. Slower end-to-end tests can still provide value, but they should not unnecessarily delay every developer feedback cycle.

The team should also address flaky tests rather than repeatedly rerunning them until they pass. A test that fails unpredictably eventually becomes noise, and developers stop trusting it. The course material notes that a good test suite should be fast, reliable, and meaningful because developer confidence is one of the major benefits of automated testing (Krasso, 2026).

This directly supports Flow and Feedback. Developers can make smaller changes and know quickly whether they introduced a regression instead of discovering the problem later during staging or release.

5. Connect to the Three Ways

These recommendations support all Three Ways. Flow improves when smaller changes can move through a reliable build, test, and deployment process without unnecessary waiting or manual intervention. Faster tests and automated staging deployments remove bottlenecks that currently slow the team down.

Feedback improves when every change automatically produces useful information. A developer should quickly know whether the application builds, whether tests pass, and whether the application can be deployed successfully. Pipeline failures should provide a clear signal rather than an unpredictable interruption.

Continuous Learning comes from using pipeline results to improve the system over time. HarborView should review recurring build failures, flaky tests, slow pipeline stages, and deployment problems to identify patterns and determine where the process needs improvement.

I would not try to solve HarborView's problems simply by adding more pipeline stages. The immediate goal should be to make the existing pipeline trustworthy. Stable builds, fast and reliable tests, repeatable deployments, and clear ownership of failures create the foundation for more advanced automation later. Once developers trust the pipeline, CI/CD becomes less of an obstacle and more of the feedback system it is intended to be.

Reference

Krasso, Richard (2026). WEB 430 Week 3 readings. Bellevue University.

Part II: AI Pipeline Review

For this part of the assignment, you will work with a simple CI/CD pipeline snippet and use an AI-assisted tool to:

- Explain what the pipeline does
- Identify potential issues
- Suggest improvements

This gives you practical experience interpreting pipelines—something you'll need for the **Final Project (Week 9)**.

My AI Prompts

I used **DeepSeek** as the AI tool for Part 2 of this assignment.

My prompts:

1. Explain this GitHub Actions CI/CD pipeline in plain language. Describe what triggers it and what each step does.

2. Review this GitHub Actions pipeline and identify potential issues or risks. Focus on CI reliability, testing, dependency management, environment consistency, and deployment gaps.

3. Suggest practical improvements to this GitHub Actions pipeline. Keep the recommendations appropriate for a small development team and focus on improving Flow, Feedback, reliability, and low-risk releases.

```
name: CI Pipeline
on:
push:
branches: [ "main" ]
jobs:
build:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3
- name: Install dependencies
run: npm install
- name: Run tests
run: npm test
- name: Build project
run: npm run build
```

AI Response

GitHub Actions CI/CD Pipeline Analysis

Plain Language Explanation

What Triggers It?

- Trigger: The pipeline runs automatically whenever code is pushed to the main branch (e.g., when a developer merges a pull request or pushes directly).

What It Does (Step-by-Step)

Step	What Happens	Purpose
Checkout Code	Pulls the latest code from the repository	Gives the runner access to your source files
Install Dependencies	Runs npm install	Downloads all packages listed in package . j son (including dev dependencies)
Run Tests	Executes npm test	Verifies your code works as expected by running the test suite
Build Project	Runs npm run build	Compiles/transpiles the code into production-ready artifacts (e.g., minified JS, csst etc.)

In Simple Terms

"Whenever someone pushes to main, GitHub spins up a fresh Ubuntu machine, grabs the code, installs its dependencies, runs the tests, and builds the project. If any step fails, the pipeline stops and the developer gets notified."

Potential Issues & Risks

Cl Reliability

- No caching — Every run re-downloads all dependencies, making runs slow and wasteful.

- No npm ci — npm install can behave differently across runs (e.g., resolving newer versions). npm ci is deterministic.
- No timeout limits — A hanging test could block the pipeline indefinitely.

Testing Gaps

- No test coverage reporting — You don't know if tests actually cover critical code paths.
- No test artifacts — Failed test output isn't saved for later inspection.
- No parallelization — Tests run sequentially, wasting time.

Dependency Management

- No lockfile enforcement— If package—lock. j son exists, npm install may ignore it in some configurations.
- No vulnerability scanning — Outdated/deprecated packages go unnoticed.

Environment Consistency

- ubuntu—latest is a moving target — GitHub can update the image, breaking builds unexpectedly.
- No Node version pinning — The default Node version may change, causing inconsistent behavior.

Deployment Gaps

- No deployment step — The pipeline stops at build; there's no release, staging, or production deployment.
- No environment separation — Can't test against staging before production.
- No rollback strategy — If a build is broken, there's no automatic way to revert.

Other Issues

- No conditional steps — Everything runs on every push, even documentation-only changes.
- No secrets management — Implicitly relies on repo secrets but doesn't demonstrate their use.

Practical Improvements 💡

Quick Wins (Low Effort, High Impact)

Improvement	Why	Implementation
Use npm ci instead of npm install	Deterministic installs from lockfile	*Change* run: npm install -> *run: npm ci*
Pin Node version	Consistent runtime	Add `node—version: 18` (or your project's version)
Add caching	Faster runs	Use *actions/ setup—node* with *cache:'npm'*
Pin runner version	Avoid surprise updates	Use `runs—on: ubuntu—22.04` instead of `ubuntu—latest`

Testing & Feedback Improvements

```
# Add test coverage and artifacts
- name: Run tests with
coverage run: npm test -- -
-coverage

- name: Upload test results
if: always() uses:
actions/upload-artifact@v3
with:
 name: test-results
 path: coverage/
```

Better Flow & Reliability

```
# Add environment-based deployment (example)
- name: Deploy to staging if:
 github.ref   'refs/heads/main'
 run: I
  # Your deploy script here
  # e.g., npm run
 deploy:staging env:
  DEPLOY_KEY: ${{ secrets.STAGlNG_KEY }}
```

Recommended Full Pipeline Structure

```
name:  Cl  Pipeline
on:
 push:
  branches:   [   'imain"   J
 pull_request:
  branches: [ 'imain" J
jobs:
 build:
  runs-on:    ubuntu-22.04
  steps:
    - uses:
    actions/checkout@v3

    - name:  Setup  Node  uses:
    actions/setup-node@v3
    with:
     node-version:      18
     cache: 'npm'

    - name:            Install
    dependencies run: npm ci

    - name: Run tests run: npm
    test

    - name: Build project run:
    npm run build

    - name:  Deploy  to  staging
    if:     github.event_name
    'push'    &&    github.ref
    'refs/heads/main'    run:
    npm   run   deploy:staging
    env:
   STAGING_TOKEN: secrets.STAGlNG_TOKEN}}
```

🎯 Priority Recommendations for a Small Team

1. Switch to npm ci + pin Node version (5 min effort, big reliability win)
2. Add caching (saves minutes per run)
3. Add a staging deployment step (even a simple one — gives you a place to verify)
4. Run tests on pull requests too (not just main) — catch issues before merging
5. Add basic test coverage reporting — helps you know what's tested

---

🏁 Final Thoughts

This pipeline is a solid starting point — it covers the basics (install, test, build). The main gaps are determinism (use npm ci), speed (add caching), and deployment (no release step). For a small team, I'd suggest implementing the quick wins first, then gradually adding deployment and coverage reporting as confidence grows.

Remember: The goal is fast feedback with low risk — every improvement should move you toward that!

Reflection

I thought DeepSeek did a good job of explaining the pipeline in plain language and breaking down what each step was accomplishing. I especially liked that it recognized the workflow was providing basic Continuous Integration but did not contain an actual deployment step, despite being described as a CI/CD pipeline. Its recommendations to run the workflow on pull requests, explicitly configure the Node.js version, use `npm ci`, and add caching were all practical improvements for making the pipeline more predictable and providing faster feedback. The recommendation to use `npm ci` was particularly useful because it is intended for automated environments and performs a clean installation based on the committed lock file. I also agreed with its suggestion to add an automated staging deployment as a logical next step toward Continuous Delivery.

However, I would not accept all of the AI's recommendations without verifying them. DeepSeek stated that a hanging test could block the pipeline indefinitely, but GitHub Actions has a default job timeout, so that statement was not completely accurate. It also generated an example using an older version of the GitHub artifact action, demonstrating that AI-generated technical recommendations can sometimes be outdated. Some suggestions, such as coverage reporting, parallelization, vulnerability scanning, and conditional execution, may be valuable, but I would introduce them only when they solve an actual problem rather than adding complexity simply because the pipeline supports it. Overall, I think DeepSeek would be useful as a first-pass pipeline reviewer because it quickly identified reliability, performance, testing, and deployment concerns that an engineer could investigate further. This exercise reinforces how I see AI-assisted DevOps in general: AI can reduce the time required to analyze a pipeline and surface potential improvements, but the engineer still needs to understand the environment, verify the recommendations, and decide which changes provide value.